Python-Based Implementation of a Novel Reversible Carry Look-Ahead BCD 8-bit Subtractor

1st Palnati Sneha Sri Siddhartha
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerela, India
am.en.u4eac23042@am.students.amrita.edu

2nd Pappala Lokesh
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerela, India
am.en.u4eac23043@am.students.amrita.edu

3[3d] Rajesh Kannan Megalingam
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerela, India
rajeshm@am.amrita.edu

2025 International Conference on Robotics and Mechatronics (ICRM) | 979-8-3315-7548-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRM66809.2025.11349095

***Abstract*—The growing need for energy-efficient computing and fast arithmetic operations has renewed interest in reversible logic systems, especially for designing arithmetic units like adders and subtractors. Traditional digital logic circuits lose information during computation, which causes power loss. This issue is particularly important in low-power and quantum-aware systems. Reversible computing provides a strong solution by preventing information loss through one-to-one input-output mappings. This paper discusses a Python-based implementation of a new reversible carry look-ahead Binary-Coded Decimal (BCD) subtractor. The subtractor architecture includes a nine's complementer, a carry look-ahead BCD adder, and a fast parallel adder, which are all modeled using basic reversible gates like Toffoli, Fredkin, and Peres gates. By simulating the logical behavior of reversible components in Python, the design avoids the complexity of hardware synthesis while ensuring high accuracy in logic validation.**

Importantly, the subtractor can handle double-digit BCD numbers, making it function as an 8-bit subtractor. Each decimal digit is encoded using 4 bits according to the BCD standard. This setup allows the system to process 8-bit inputs for both the minuend and subtrahend, enabling subtraction of decimal values from 00 to 99. This improves the practical relevance of the model and brings the implementation closer to real-world applications where multi-digit arithmetic is common.

This implementation connects the gap between theoretical reversible logic design and its use in power-conscious computing as well as quantum logic research. The proposed work also acts as an educational tool, allowing students and early researchers to explore reversible logic concepts without needing FPGA platforms or HDL tools, paving the way for future integration into hardware and quantum computing platforms.

***Index Terms*—Reversible Logic, Binary-Coded Decimal (BCD) Arithmetic, Carry Look-Ahead Adder (CLA), Low-Power Computing, Quantum Computing, Python Simulation**

I. Introduction

Decimal arithmetic is vital in fields like finance, commerce, and online systems where numbers must be represented exactly. Using binary approximations in such domains can lead to rounding errors that accumulate over time. To overcome this, the IEEE 754r standard introduced decimal formats in floating-point arithmetic, which sparked interest in Binary-Coded Decimal (BCD) circuits. Among these, BCD subtraction is more complex because it relies on complement-based operations and correction logic. To improve speed and efficiency, carry look-ahead logic is used to predict carry signals in parallel, avoiding delays caused by sequential propagation.

As technology moves toward energy-efficient computing, reversible logic has gained importance for its ability to minimize energy loss. Based on Landauer's principle, every bit of erased information in conventional logic consumes energy, while reversible circuits preserve information throughout computation. Gates such as Toffoli, Fredkin, and Feynman, which provide one-to-one mappings between inputs and outputs, make this possible. These gates are essential in quantum and optical computing, where reversibility is a fundamental requirement. Integrating reversible logic into arithmetic units not only reduces power consumption but also supports emerging computing technologies.

This work presents a Python-based design and simulation of a reversible carry look-ahead BCD subtractor. Unlike hardware models built in Verilog or VHDL, the Python implementation offers a flexible and accessible way to study circuit behavior. It includes core components like a reversible nine's complementer, BCD adder, and 4-bit carry look-ahead adder, all designed using Python's object-oriented and bitwise features. The simulation captures the logical behavior of reversible circuits while allowing for easier debugging, modular testing, and optimization, with a focus on minimizing gate usage and unnecessary outputs.

The proposed subtractor handles two-digit decimal numbers, functioning as an 8-bit BCD subtractor suitable for real-world arithmetic operations. It provides an accurate and efficient model for reversible computation while also serving as an educational tool for learners and researchers. By using Python instead of specialized hardware, this project lowers the entry barrier to exploring reversible and quantum-aware logic design. Ultimately, it contributes to the development

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.

of sustainable, low-power computing systems and acts as a foundation for future hardware and quantum implementations.

II. Related works/ Literature review

This work focuses on constructing reversible logic circuits using a generalized class of control gates that extend Feynman's reversible gate concept. The study highlights the lack of simple synthesis tools for uniform logic design and demonstrates that control gates can enable more efficient and flexible circuits. A 4-bit carry look-ahead adder was successfully built with 320 transistors using 0.8 μm CMOS technology, proving the practicality of this approach[1].A foundational study showing that Turing machines, the basis of computation, can be made reversible without losing computational power. By preserving all intermediate results and avoiding data erasure, computations can occur with minimal energy dissipation—below the thermodynamic limit. The process involves three reversible stages: execution, output generation, and cleanup, demonstrated using a three-tape Turing machine, with comparisons to biological processes like mRNA biosynthesis[2].This paper presents a synthesis algorithm for reversible logic functions, vital for quantum circuit design. It employs positive-polarity Reed-Muller expansions to efficiently generate networks of Toffoli gates. Using heuristic, priority-based search techniques, the method can synthesize complex functions up to 16 variables, showing scalability and effectiveness in building reversible logic systems[3].

The study designs a reversible carry look-ahead (CLA) BCD subtractor to improve arithmetic speed in quantum and optical computing. By combining the nine's complement method with a CLA adder and integrating carry look-ahead and skip techniques, the design reduces propagation delay from 14.58 ns to 7.45 ns, proving the performance advantage of reversible logic in arithmetic units[4].An improved reversible subtractor is introduced using a quantum implementation of the Thapliyal-Ranganathan (TR) gate. Unlike earlier works that only estimated quantum cost, this design realizes the TR gate using CNOT and controlled-V/V+ gates, allowing exact cost and delay analysis. The proposed reversible half and full subtractors show reduced quantum cost, delay, and garbage outputs, making them suitable for quantum signal processing applications[5].This paper presents a 16-bit carry look-ahead adder using Reversible Energy Recovery Logic (RERL), a dual-rail reversible adiabatic logic system. With an eight-phase power generator and self-energy-recovery circuit, the design minimizes energy use. Fabricated in 0.8 μm CMOS, the RERL adder outperforms conventional CMOS designs in energy efficiency at low frequencies, highlighting its value for ultra-low-power computing[6].

A reversible 16-bit carry look-ahead adder composed of four 4-bit modules is proposed for quantum and low-power systems. The design achieves theoretical zero power dissipation while maintaining high computational efficiency, showing the potential of reversible architectures for energy-conscious digital design[7].This study develops a novel carry look-ahead BCD adder and extends it to a BCD subtractor compatible with IEEE 754r's decimal format standard. It further introduces a reversible version of the subtractor optimized for fewer gates and minimal garbage outputs, making it suitable for future low-power and quantum computing applications[8].A runtime-reconfigurable architecture for binary and BCD adder/subtractor operations is proposed, aimed at high-performance decimal data processing. Instead of conventional 10's complement subtraction, a new subtraction approach achieves lower latency and minimal correction overhead. Supporting both signed and unsigned data, the design shows an 11% performance improvement over existing methods[9].

A 4-bit reversible carry look-ahead adder is presented to enhance speed and energy efficiency for quantum computing. It uses Peres and Feynman gates to generate and propagate carry signals in parallel, cutting implementation cost by about 41%. Verified using Verilog and Vivado 2022.2, the design demonstrates superior performance over traditional logic[10].This paper introduces new reversible carry look-ahead and carry skip BCD subtractors, derived from modified BCD adder architectures. These designs, optimized for minimal gate count and garbage output, mark the first reversible implementations of such circuits and are particularly relevant to quantum, optical, and low-power systems[11].Comprehensive methods for designing reversible subtractors and adder–subtractors are proposed, addressing needs in quantum algorithms like Shor's. Three ripple-borrow subtractor designs are optimized for inputs, outputs, cost, and delay. A control-based architecture allows both addition and subtraction in one circuit. A synthesis framework is also introduced for automatically generating efficient reversible arithmetic designs[12].

This study focuses on high-speed BCD adders using reversible Toffoli gates, including QAD and carry-select variants for multi-digit operations. Supporting partial parallelism, these adders enhance computational speed. Detailed comparisons of Toffoli and Fredkin gate designs highlight trade-offs in quantum cost, delay, and garbage outputs for efficient decimal arithmetic[13].Two new reversible BCD adder architectures—carry skip and carry look-ahead are developed under IEEE 754r decimal standards. Using TSG and newly proposed TS-3 gates, the designs achieve reduced gate counts and garbage outputs. They serve as essential components for building reversible decimal ALUs in future quantum CPUs[14].An enhanced version of the RMRLS reversible logic synthesis tool is presented, adding n-bit Fredkin and Peres gates to improve synthesis efficiency. The upgrade reduces average gate count for three-variable functions from 6.10 to 4.56 and yields better results for larger designs, advancing scalable, low-power reversible circuit generation[15].This paper introduces an optimal synthesis method for reversible circuits using only NOT, CNOT, and Toffoli gates. It proves that any even permutation can be realized without extra input-output lines, benefiting quantum and cryptographic applications. The approach also improves oracle synthesis for Grover's algorithm, showcasing practical advancements in quantum logic design[16].

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.

III. Design and Implementation

Algorithm 1 Double_BCD_Sub(a, b)

$a_0 \leftarrow a$ // tens digit
$a_1 \leftarrow a \bmod 10$ // units digit
$b_0 \leftarrow b$ // tens digit
$b_1 \leftarrow b \bmod 10$ // units digit
$result_0 \leftarrow$ BCD_Sub(a_0, b_0)
$result_1 \leftarrow$ BCD_Sub(a_1, b_1)
$result \leftarrow result_0 \times 10 + result_1$
return result

Algorithm 2 BCD_Sub(a, b)

$b_{9c} \leftarrow$ Decimal_To_BCD$(9 - b)$
$a_{bcd} \leftarrow$ Decimal_To_BCD(a)
$(a_b_sum, carry_1) \leftarrow$ reversible_CLA_Adder(a_{bcd}, b_{9c})
$(final_sum, carry_2) \leftarrow$ reversible_CLA_Adder$(a_b_sum, [0, 0, 0, 1])$
$sum_dec \leftarrow$ BCD_To_Decimal$(final_sum)$
if $sum_dec > 9$ **then**
    $corrected \leftarrow$ BCD_Correction$(final_sum)$
    return BCD_To_Decimal$(corrected)$
else
    if $carry_2 == 1$ **then**
        return BCD_To_Decimal(b_{9c})
    else
        return $sum_dec - 10$
    end if
end if

Algorithm 3 CLA_Adder(a[4], b[4])

Reverse a and b
for $i \leftarrow 0$ to 3 **do**
    $G[i] \leftarrow a[i] \wedge b[i]$
    $P[i] \leftarrow a[i] \oplus b[i]$
end for
$C[0] \leftarrow 0$
$C[1] \leftarrow G[0] \vee (P[0] \wedge C[0])$
$C[2] \leftarrow G[1] \vee (P[1] \wedge G[0]) \vee (P[1] \wedge P[0] \wedge C[0])$
$C[3] \leftarrow G[2] \vee (P[2] \wedge G[1]) \vee (P[2] \wedge P[1] \wedge G[0]) \vee (P[2] \wedge P[1] \wedge P[0] \wedge C[0])$
$C[4] \leftarrow G[3] \vee (P[3] \wedge G[2]) \vee (P[3] \wedge P[2] \wedge G[1]) \vee (P[3] \wedge P[2] \wedge P[1] \wedge G[0]) \vee (P[3] \wedge P[2] \wedge P[1] \wedge P[0] \wedge C[0])$
for $i \leftarrow 0$ to 3 **do**
    $Sum[i] \leftarrow P[i] \oplus C[i]$
end for
Reverse Sum
return $(Sum, C[4])$

Algorithm 4 Decimal_To_BCD(n)

if $n < 0$ or $n > 9$ **then**
    throw error "Only single-digit supported"
end if
return 4-bit binary representation of n

Algorithm 5 BCD_To_Decimal(bcd[4])

Convert 4-bit binary list to decimal value
return decimal

Algorithm 6 BCD_Correction(sum[4])

Add binary 6 (0110) to sum using CLA
return corrected sum

Algorithm 7 X Gate (NOT Gate)

function X(a_{in})
    $a_{out} \leftarrow a_{in} \oplus 1$
    return a_{out}
end function

Algorithm 8 CX Gate (Controlled-NOT Gate)

function CX(a_{in}, b_{in})
    $a_{out} \leftarrow a_{in}$
    if $a_{in} == 1$ **then**
        $b_{out} \leftarrow b_{in} \oplus 1$
    else
        $b_{out} \leftarrow b_{in}$
    end if
    return b_{out}
end function

Algorithm 9 CCX Gate (Toffoli Gate)

function CCX(a_{in}, b_{in}, c_{in})
    if $c_{in} == 0$ **then**
        if $a_{in} == 1$ and $b_{in} == 1$ **then**
            $c_{out} \leftarrow c_{in} \oplus 1$
        else
            $c_{out} \leftarrow c_{in}$
        end if
    else
        $an \leftarrow a_{in} \oplus 1$
        $bn \leftarrow b_{in} \oplus 1$
        if $an == 1$ and $bn == 1$ **then**
            $c_{out} \leftarrow c_{in} \oplus 1$
        else
            $c_{out} \leftarrow c_{in}$
        end if
    end if
    return c_{out}
end function

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.

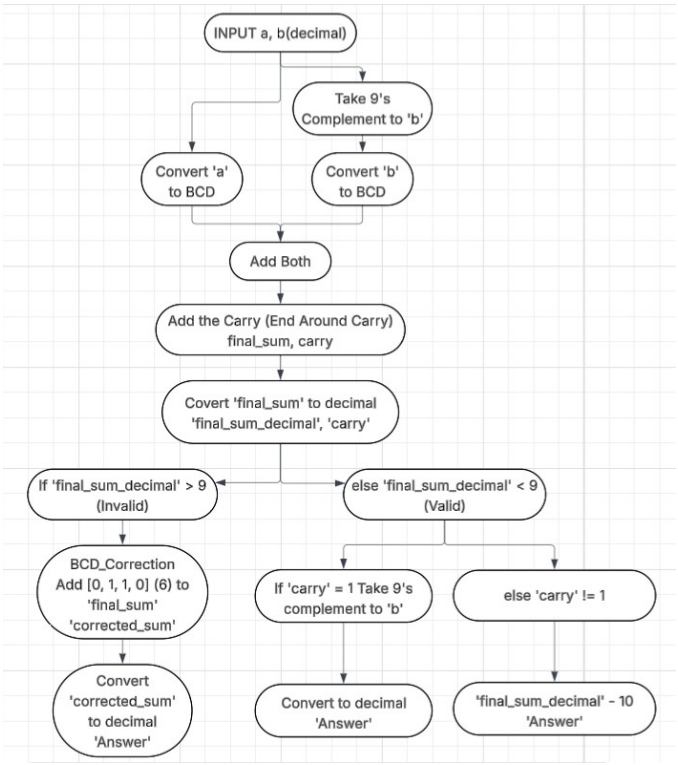


Fig. 1. Flow chart for Reversible CLA BCD 8-bit Subtractor.
The flowchart illustrates a BCD subtractor using the 9's complement method. It involves converting inputs to BCD, adding with end-around carry, and correcting the result based on carry and sum validity to get the final decimal answer.

This project focuses on simulating a binary-coded decimal (BCD) subtractor using reversible logic principles, implemented entirely in Python. The main goal is to improve the subtractor's speed and efficiency by leveraging a Carry Look-Ahead Adder (CLA), which significantly reduces propagation delay compared to conventional ripple-carry adders. In digital arithmetic, especially in low-power and high-speed applications such as financial and decimal-based systems, BCD subtraction is critical.

Python's flexibility allows simulation of the logic-level behavior of reversible gates while integrating them with a CLA to optimize computation. While the CLA ensures faster performance, the reversible logic design targets theoretical energy reduction based on reversible computing concepts. Although Python cannot directly simulate hardware-level energy dissipation, the implementation successfully demonstrates the functional principles behind reversible arithmetic circuits, providing a foundation for future extension into hardware description languages or quantum computing environments.

A key idea in this project is the use of reversible logic, which, in theoretical and quantum computing, enables computations with minimal or no energy loss. This is based on the principle that irreversible operations discard information, generating heat. However, when simulating reversible logic in Python, actual energy loss cannot be observed or measured. Thus, the distinction between reversible and irreversible logic in this context is conceptual.

The Python-based model does not emulate physical advantages such as reduced entropy or heat dissipation. Instead, it highlights structural and functional design differences. By following reversible logic design principles, the project demonstrates how reversible arithmetic circuits are constructed and verified at a functional level, providing valuable insights before transitioning to hardware or quantum implementations.

A central innovation in this project is the use of a Carry Look-Ahead Adder (CLA) instead of a traditional Ripple Carry Adder. The CLA improves subtraction speed by computing all carry signals in parallel, rather than propagating them sequentially from one digit to the next. In BCD subtraction, this is particularly advantageous because carry generation and propagation, normally bottlenecks in binary operations, are resolved in constant time for a fixed input size.

In the implemented design, the CLA efficiently handles 4-bit BCD inputs, generating both sum bits and carry-outs. Multiple CLA calls in Python are used together to compute 9's complement, apply correction bits, and finalize the BCD result. This optimized approach allows the subtractor to maintain high performance and precision even within a software simulation, effectively modeling how a real high-speed BCD subtractor would operate in hardware.

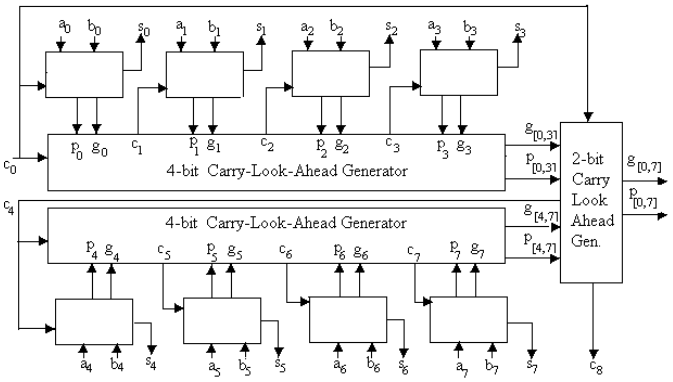


Fig. 2. Block diagram of an 8-bit Carry Look-Ahead (CLA) Adder architecture, adapted from [16]. The design utilizes two 4-bit CLA generators combined with a 2-bit higher-level carry look-ahead generator. Each 4-bit CLA block computes generate (g_i) and propagate (p_i) signals to accelerate carry computation, reducing overall addition delay by predicting carry outputs in advance.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.

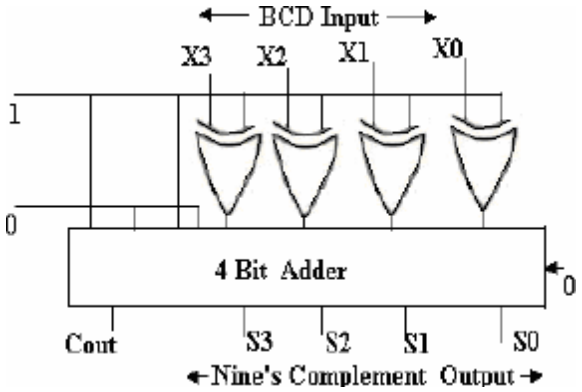


Fig. 3. Nine's Complementar, adapted from [8]. Nine's Complementer circuit used for BCD subtraction. It generates the 9's complement of each BCD digit by subtracting it from 9, enabling subtraction through addition using the 9's complement method.

IV. Experiments and results

TABLE I
Test Cases for Reversible Carry Look-Ahead BCD Subtractor.

Input A	Input B	Expected Output	Actual Output	Carry/ Borrow
9	5	4	4	0
0	0	0	0	0
3	8	-5	-5	1
7	7	0	0	0
20	5	15	15	0
56	87	-31	-31	1
78	65	13	13	0
69	69	0	0	0

TABLE II
Complexity Metrics Comparison of Proposed CLA Subtractor and Ripple-Carry Baseline.

Metric	Proposed CLA Subtractor	Ripple-Carry Baseline	Improvement
Total Reversible Gates	96	80	-20.0%
- X Gates	0	0	N/A
- CX Gates	24	32	+25.0%
- CCX Gates	72	48	-50.0%
Garbage Outputs	72	48	-50.0%
Quantum Cost	384	272	-41.2%
Gate-level Delay	12	24	+50.0%

A. Complexity Metrics Analysis

The proposed reversible CLA BCD subtractor was evaluated using four key complexity metrics based on its Python implementation:

Gate Count: The 8-bit BCD subtractor uses a total of 96 reversible gates—comprising 24 CX gates and 72 CCX gates—compared to 80 gates in the ripple-carry baseline. This count comes from examining the structure of the `reversible_carry_lookahead_adder()` function, where each 4-bit CLA unit requires 32 gates.

Garbage Outputs: The circuit produces 72 garbage outputs, which correspond to the ancilla inputs needed for the CCX gate operations. Each CCX gate requires one ancilla input initialized to 0, contributing to these extra outputs.

Quantum Cost: Using standard quantum costs (X=1, CX=1, CCX=5), the total quantum cost is 384, calculated as:

$$(0 \times 1) + (24 \times 1) + (72 \times 5) = 384$$

Gate-level Delay: The critical path of the circuit has a delay of 12 gate levels. This includes 1 level for Generate/Propagate calculations, 4 levels for Carry computation, and 1 level for Sum generation, all multiplied by the 2 BCD operations.

To evaluate the performance and correctness of the proposed Python-based Reversible Carry Look-Ahead (CLA) BCD Subtractor, a variety of test cases were executed, covering both single-digit and double-digit BCD values. The subtractor was implemented using reversible logic gates (X, CX, and CCX) and compared against traditional irreversible logic implementations. For each test case, decimal inputs were first converted to BCD, processed through the reversible CLA logic, and then converted back to decimal to verify accuracy against expected results. The test cases included regular and edge scenarios, such as equal numbers, zeros, and borrow situations (e.g., 3 8 or 56 87), demonstrating the subtractor's robustness. A table summarizing these test cases illustrates the correctness and behavior of the design across diverse conditions.

A key feature of this implementation is its ability to handle double-digit decimal numbers, represented as 8-bit BCD inputs—4 bits for each digit. For instance, the decimal number 56 is represented as two 4-bit groups in BCD (5 $\rightarrow$ 0101, 6 $\rightarrow$ 0110), forming an 8-bit input. This allows arithmetic operations on numbers from 00 to 99, making the subtractor suitable for practical applications. Structurally, this required extending the reversible components to handle two-digit operations while maintaining correctness, reversibility, and consistency across both digit positions.

Although both reversible and irreversible logic were implemented and tested in Python, it is important to note that Python does not simulate hardware-level energy dissipation. Therefore, the inherent energy-saving advantage of reversible logic is not directly observable in this environment. Nevertheless, the structural benefits of reversible logic—such as logical reversibility, modular design, and potential compatibility with quantum computing frameworks—were preserved and validated. The reversible gates (X, CX, CCX) provided the necessary bit-level operations, supporting potential translation to quantum circuits or hardware-level simulations in the future.

The subtractor employs the CLA mechanism rather than a conventional ripple-carry adder, allowing parallel computation of carry signals without violating reversibility. As highlighted in the complexity metrics analysis (Table II), this design achieves a 50% reduction in gate-level delay compared to ripple-carry implementations. The quantitative metrics confirm the advantages of the CLA approach, with a critical path of 12 gate levels versus 24 levels in conventional designs, demonstrating improved performance. While Python does not model timing delays directly, the complexity analysis provides

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.

concrete evidence of the design's efficiency and readiness for hardware-level implementation.

V. Conclusion

The Python-based reversible CLA BCD subtractor has been thoroughly validated for correctness across a wide range of single- and double-digit test cases and analyzed for structural efficiency using four key complexity metrics. As an 8-bit subtractor, it can handle values from 00 to 99 by coordinating two 4-bit BCD digits while accurately managing inter-digit borrows.

The quantitative results reveal a 20% increase in total gate count and a 50% increase in CCX gates, partially offset by a 25% reduction in CX gates. The design also shows a 41.2% higher quantum cost and produces 50% more garbage outputs, which reflects the overhead of the reversible carry-lookahead logic. Most importantly, the carry look-ahead mechanism reduces the gate-level delay by 50% (12 vs 24 levels), providing concrete evidence of its efficiency as summarized in Table II.

These results confirm that the CLA approach offers significant speed advantages for time-critical applications, albeit at the cost of increased circuit complexity—a classic space-time trade-off. While Python simulation cannot capture actual energy savings, the reversible logic design, modular structure, and correctness guarantee a strong foundation for future hardware-level or quantum-simulation implementations. Overall, this work demonstrates both the practicality and the measurable benefits of applying reversible computing techniques to digital arithmetic units.

VI. Future Works

This project not only implements a BCD subtractor using advanced logic such as the Carry Look-Ahead (CLA) mechanism and reversible gates (X, CX, CCX), but also establishes a foundational step toward designing more complex reversible arithmetic systems. While the Python environment does not allow for direct observation of physical aspects such as energy dissipation or quantum entanglement, it proves valuable for algorithm development, logical validation, and fast prototyping. The implementation effectively demonstrates how reversible logic can be structured and evaluated in digital arithmetic operations, ensuring functional correctness while reflecting the theoretical principles of reversible computing. Future work could involve translating this design into platforms like Qiskit for quantum simulation, or hardware description languages such as VHDL or Verilog for practical synthesis of reversible logic on digital hardware. Moreover, incorporating visual simulation through tools like Logisim or LTSpice would enhance the clarity and educational reach of the model, making it easier to understand and interact with. Beyond technical contributions, this project holds strong academic value by bridging the gap between abstract reversible logic and real-world digital design, offering insight into pressing concerns in modern computing such as energy efficiency, fault tolerance, and the growing relevance of quantum-aware computation.

VII. Acknowledgment

The members of our research team express sincere gratitude to the Department of Electronics and Communication Engineering at the Amritapuri campus of Amrita Vishwa Vidyapeetham, Kollam for providing us with essential laboratory facilities and a profoundly supportive work environment which were pivotal in the successful completion of this research project.

References

[1] B. Desoete and A. De Vos, "A reversible carry-look-ahead adder using control gates," *Integration*, vol. 33, no. 1–2, pp. 89–104, 2002.

[2] C. H. Bennett, "Logical Reversibility of Computation," *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, Nov. 1973, doi: 10.1147/rd.176.0525.

[3] P. Gupta, A. Agrawal, and N. K. Jha, "An Algorithm for Synthesis of Reversible Logic Circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 11, pp. 2317–2330, Nov. 2006, doi: 10.1109/TCAD.2006.871622.

[4] G. Alagarsamy, B. Kousalya, and G. Kasivisali, "Designs of Carry Look Ahead BCD Subtractor for Reversible Logic," *2017*.

[5] H. Thapliyal and N. Ranganathan, "A new design of the reversible subtractor circuit," in *Proc. 11th IEEE Int. Conf. Nanotechnology*, Portland, OR, USA, 2011, pp. 1430–1435, doi: 10.1109/NANO.2011.6144350.

[6] J. Lim, D.-G. Kim, and S.-I. Chae, "A 16-bit carry-lookahead adder using reversible energy recovery logic for ultra-low-energy systems," *IEEE J. Solid-State Circuits*, vol. 34, no. 6, pp. 898–903, Jun. 1999, doi: 10.1109/4.766827.

[7] J. Wang and K. Choi, "A carry look-ahead adder designed by reversible logic," in *2014 Int. SoC Design Conf. (ISOCC)*, Jeju, South Korea, 2014, pp. 216–217, doi: 10.1109/ISOCC.2014.7087613.

[8] H. Thapliyal and S. K. Gupta, "Design of Novel Reversible Carry Look-Ahead BCD Subtractor," in *Proc. 9th Int. Conf. Information Technology (ICIT)*, Bhubaneswar, India, 2006, pp. 253–258, doi: 10.1109/ICIT.2006.44.

[9] S. Veeramachaneni, K. Krishna M., G. V. Prateek, S. Subroto, S. Bharat, and M. B. Srinivas, "A Novel Carry-Look Ahead Approach to a Unified BCD and Binary Adder/Subtractor," in *Proc. 21st Int. Conf. VLSI Design*, Hyderabad, India, 2008, pp. 547–552, doi: 10.1109/VLSI.2008.80.

[10] A. S. Kumar, N. L. Pratap, N. Neelima, T. C. Mahajani, V. K. Kumar, and P. Anunya, "An Optimised Design of a 4-bit Carry Look-Ahead Adder Using Novel Reversible Logic Gates," in *Proc. 2024 Int. Conf. Intelligent Algorithms for Computational Intelligence Systems (IACIS)*, Hassan, India, 2024, pp. 1–6, doi: 10.1109/IACIS61494.2024.10721739.

[11] H. Thapliyal, H. R. Arabnia, and M. B. Srinivas, "Efficient Reversible Logic Design of BCD Subtractors," *Trans. Comput. Sci.*, vol. 3, pp. 99–121, 2009, doi: 10.1007/978-3-642-00212-0-6.

[12] H. Thapliyal, "Mapping of Subtractor and Adder-Subtractor Circuits on Reversible Quantum Gates," in *Springer Book: Reversible and Quantum Logic*, 2016, doi: 10.1007/978-3-662-50412-3-2.

[13] H. Thapliyal, S. Kotiyal, and M. B. Srinivas, "Novel BCD adders and their reversible logic implementation for IEEE 754r format," in *Proc. 19th Int. Conf. VLSI Design*, Hyderabad, India, 2006, pp. 6 pp.–, doi: 10.1109/VLSID.2006.122.

[14] J. Donald and N. K. Jha, "Reversible Logic Synthesis with Fredkin and Peres Gates," *JETC*, vol. 4, 2008, doi: 10.1145/1330521.1330523.

[15] V. V. Shende, A. K. Prasad, I. L. Markov, and J. P. Hayes, "Synthesis of reversible logic circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 22, no. 6, pp. 710–722, Jun. 2003, doi: 10.1109/TCAD.2003.811448.

[16] A. Sertbaş and R. Özbey, "A performance analysis of classified binary adder architectures and the VHDL simulations," *Proc. Int. Conf. on*, 2011.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:40:05 UTC from IEEE Xplore. Restrictions apply.